

Informática Avanzada

Bibliotecas de Clases
Documentación de Clases
Otras Colecciones

Bibliotecas de clases

- Las librerías contienen clases predefinidas
- Estas clases están agrupadas en paquetes (packages) para resolver problemas comunes
 - `java.util`
 - `java.math`
 - `java.io`
- Se puede importar todo un paquete o clases individuales en nuestro sistema
 - `import java.util.*;`
 - `import java.io.File;`
- El package ***java.lang*** contiene clases de uso habitual (e.g., `String`) y es importado por defecto



docs.oracle.com/javase/10/docs/api/index.html?overview-summary.html

← → ↺

Incógnita

java.time.chrono
java.time.format
java.time.temporal
java.time.zone
java.util
java.util.concurrent
java.util.concurrent.atomic
java.util.concurrent.locks
java.util.function
java.util.jar
java.util.logging
java.util.prefs
java.util.regex
java.util.spi
java.util.stream
java.util.zip
javafx.animation

Set
SortedMap
SortedSet
Splitter
Splitter.OfDouble
Splitter.OfInt
Splitter.OfLong
Splitter.OfPrimitive

Classes
AbstractCollection
AbstractList
AbstractMap
AbstractMap.SimpleEntry
AbstractMap.SimpleImmutableEntry
AbstractQueue
AbstractSequentialList
AbstractSet
ArrayDeque
ArrayList
Arrays
Base64
Base64.Decoder
Base64.Encoder
BitSet
Calendar
Calendar.Builder
Collections
Currency
Date
Dictionary
DoubleSummaryStatistics
EnumMap
EnumSet
EventListenerProxy
EventObject
FormatableFlags
Formatter
GregorianCalendar
HashMap

OVERVIEW MODULE PACKAGE **CLASS** USE TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS **FRAMES** NO FRAMES

SUMMARY: NESTED | FIELD | CONSTRUCTOR | METHOD DETAIL: FIELD | CONSTR | METHOD

SEARCH:

Constructor Summary

Constructors

Constructor	Description
<code>ArrayList()</code>	Constructs an empty list with an initial capacity of ten.
<code>ArrayList(int initialCapacity)</code>	Constructs an empty list with the specified initial capacity.
<code>ArrayList(Collection<? extends E> c)</code>	Constructs a list containing the elements of the specified collection, in the order they are returned by the collection's iterator.

Method Summary

All Methods Instance Methods Concrete Methods

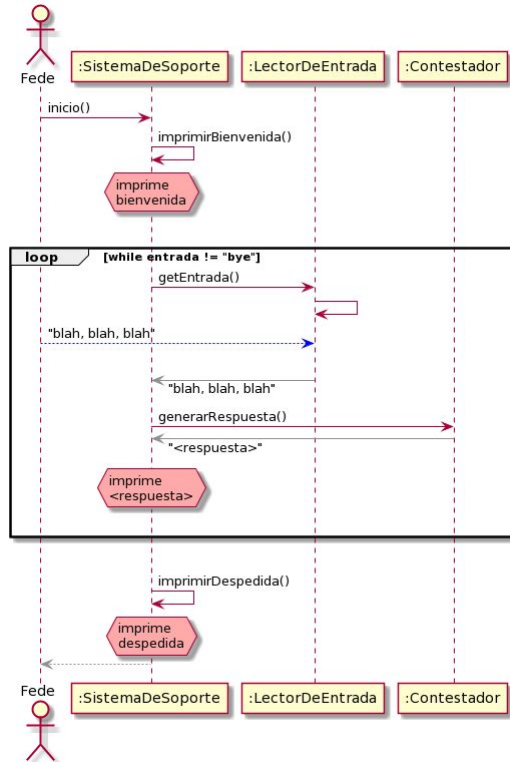
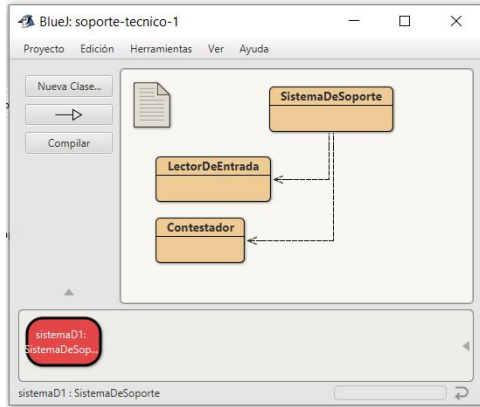
Modifier and Type	Method	Description
void	<code>add(int index, E element)</code>	Inserts the specified element at the specified position in this list.
boolean	<code>add(E e)</code>	Appends the specified element to the end of this list.
boolean	<code>addAll(int index, Collection<? extends E> c)</code>	Inserts all of the elements in the specified collection into this list, starting at the specified position.
boolean	<code>addAll(Collection<? extends E> c)</code>	Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator.
void	<code>clear()</code>	Removes all of the elements from this list.
Object	<code>clone()</code>	Returns a shallow copy of this ArrayList instance.
boolean	<code>contains(Object o)</code>	Returns true if this list contains the specified element.
void	<code>ensureCapacity(int minCapacity)</code>	Increases the capacity of this ArrayList instance, if necessary, to ensure that it can hold at least the number of elements specified by the minimum capacity argument.
void	<code>forEach(Consumer<? super E> action)</code>	Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
E	<code>get(int index)</code>	Returns the element at the specified position in this list.
int	<code>indexOf(Object o)</code>	Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.

Interfaz vs. Implementación

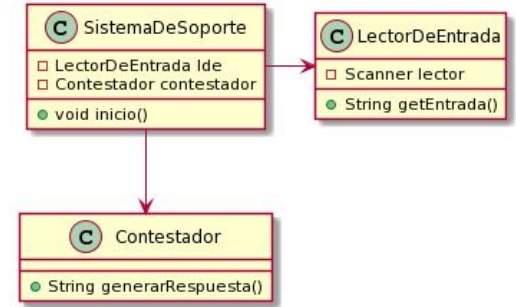
- La Interfaz hace referencia al “qué”
 - Conjunto de métodos **públicos**
 - Está resumida en la documentación
 - Nombre y descripción de la Clase
 - Nombre y descripción de los métodos **públicos**
 - Nombre y descripción de los constructores **públicos**
 - Nos permite conocer cómo **usar** la clase
- La Implementación hace referencia al “cómo”
 - Es el código fuente que implementa la funcionalidad en sí
 - Está generalmente oculta
- No es necesario conocer la implementación para usar una clase, sólo su interfaz.
- Las clases se documentan de tal manera que la interfaz pueda ser leída por sí misma (clase, comentario de método).



Proyecto Sistema Soporte Técnico



Sistema De Soporte - Class Diagram



Generar documentación de nuestras clases - javadoc

Debemos documentar nuestras propias clases incluyendo

- Para cada clase
 - Nombre de clase
 - Descripción del propósito de la clase
 - Número de versión, autores, etc. (opcionales)
- Para cada constructor y método
 - Nombre del método
 - El tipo de valor retornado
 - Lista de parámetros requeridos
 - Descripción del propósito y funcionamiento del método y valor retornado

```
/**
 * La clase Contestador representa un objeto generador de respuestas.
 * Se lo usa para generar una respuesta automática por azar seleccionando
 * una frase de una lista predefinida de respuestas.
 *
 * @version 1.0
 * @author Michael Kolling and David J. Barnes
 */
public class Contestador
{
```

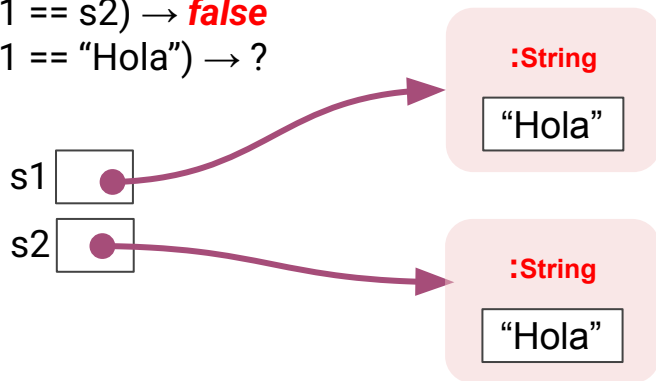
```
/**
 * Genera una respuesta desde un set de palabras de entrada dado
 *
 * @param palabras Un set de palabras ingresado por el usuario
 * @return Una cadena que se podría mostrar como respuesta
 */
public String generarRespuesta(HashSet<String> palabras)
{
    Iterator<String> it = palabras.iterator();
```

Identidad vs Igualdad

```
String s1 = "Hola";  
String s2 = "Hola";
```

$(s1 == s2) \rightarrow \text{false}$

$(s1 == \text{"Hola"}) \rightarrow ?$



$s1.equals(s2) \rightarrow \text{true!!!}$

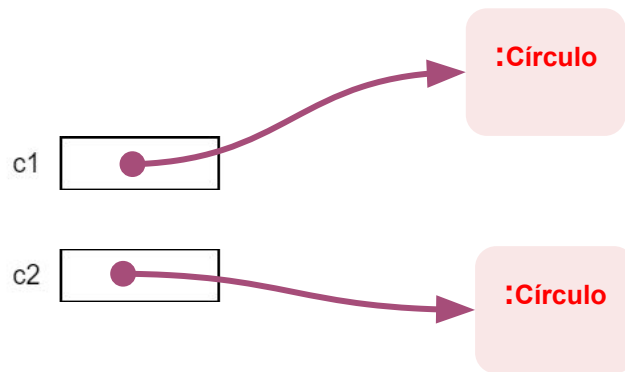
$s2.equals(s1) \rightarrow \text{true!!!}$

$s1.equals(\text{"Hola"}) \rightarrow \text{true!!!}$

```
Circulo c1 = new Circulo();
```

```
Circulo c2 = new Circulo();
```

$(c1 == c2) \rightarrow \text{false}$



Clase "Random"

```
Contestador X
[Compilar] [Deshacer] [Cortar] [Copiar] [Pegar] [Buscar...] [Cerrar] Código Fuente

1 import java.util.ArrayList;
2 import java.util.Random;
3
4 public class Contestador
5 {
6     private Random generadorDeAzar;
7     private ArrayList<String> respuestas;
8     /**
9      * Crea un Contestador
10     */
11     public Contestador()
12     {
13         generadorDeAzar = new Random();
14         respuestas = new ArrayList<String>();
15         rellenarRespuestas();
16     }
17     /**
18     * Genera una respuesta
19     *
20     * @return Una cadena que se podría mostrar como respuesta
21     */
22     public String generarRespuesta()
23     {
24         // Pick a random number for the index in the default response
25         // list. The number will be between 0 (inclusive) and the size
26         // of the list (exclusive).
27         int indice = generadorDeAzar.nextInt(respuestas.size());
28         return respuestas.get(indice);
29     }
30 }
```

docs.oracle.com/javase/8/docs/api/index.html?java/util/Random.h... Incógnito Actualizar

OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS FRAMES NO FRAMES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3
java.util

Class Random

java.lang.Object
java.util.Random

All Implemented Interfaces:
Serializable

Direct Known Subclasses:
SecureRandom, ThreadLocalRandom

public class Random
extends Object
implements Serializable

An instance of this class is used to generate a stream of pseudorandom numbers. The class uses a 48-bit seed, which is modified using a linear congruential formula. (See Donald Knuth, *The Art of Computer Programming, Volume 2, Section 3.2.1.*)

If two instances of Random are created with the same seed, and the same sequence of method calls is made for each, they will generate and return identical sequences of numbers. In order to guarantee this property, particular algorithms are specified for the class Random. Java implementations must use all the algorithms shown here

int	nextInt() Returns the next pseudorandom, uniformly distributed int value from this random number generator's sequence.
int	nextInt(int bound) Returns a pseudorandom, uniformly distributed int value between 0 (inclusive) and the specified value (exclusive), drawn from this random number generator's sequence.

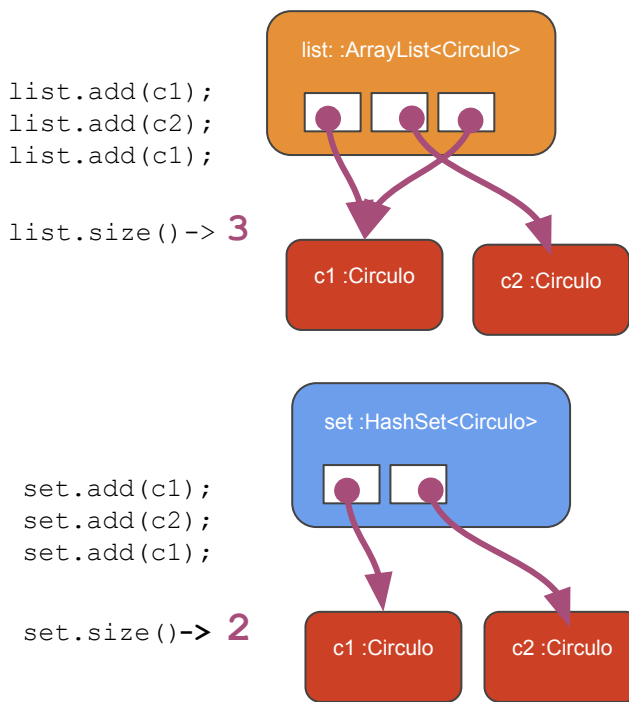
Otras Colecciones - Maps, Sets, Lists

- Java ofrece muchas colecciones, que pueden agruparse en
 - List
 - Map
 - Set
- A su vez existen diferentes implementaciones de listas, mapas y sets
 - ArrayList, LinkedList, AbstractList, ...
 - HashSet, LinkedHashSet, TreeSet,...
 - HashMap, TreeMap, LinkedHashMap, ...
- Cada implementación tiene sus propias características en cuanto a la forma de realizar las acciones, pero respetan la misma interfaz



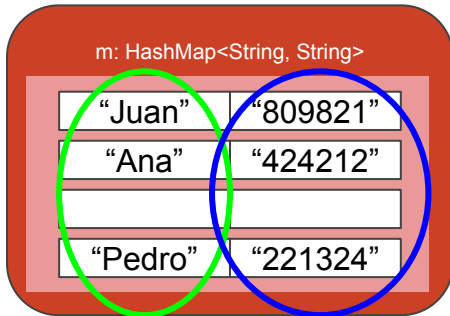
Set

- Son colecciones que agrupan elementos (similar a las listas)
- **No permiten** elementos **duplicados** (a diferencia de las listas)
 - Al agregar un elemento que ya existe, se reemplaza la entrada anterior (por el contrario, en una lista, puedo tener varias entradas apuntando el mismo objeto)
- Sus elementos no están indexados
- Pueden recorrerse a través de un **iterator** o bucle **for-each**
 - El orden en que los elementos se recorren en una iteración es “indeterminado”, pero siempre es el mismo.
- Los métodos principales son
 - ***add, remove, contains...*** (no ***get***)



Mapas

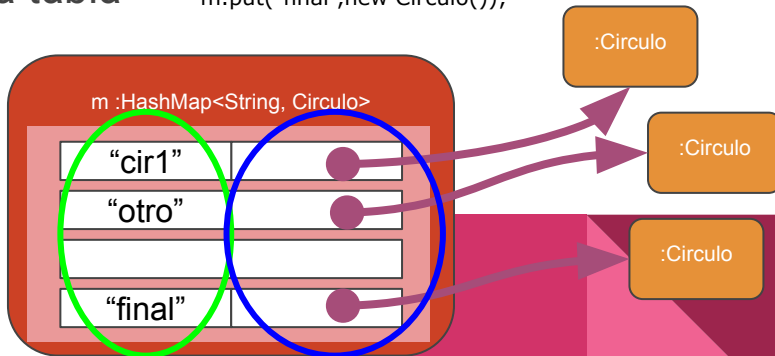
```
HashMap<String,String> m;  
m = new HashMap<String,String>();  
  
m.put("Juan","809821");  
m.put("Ana","424212");  
m.put("Pedro","221324");
```



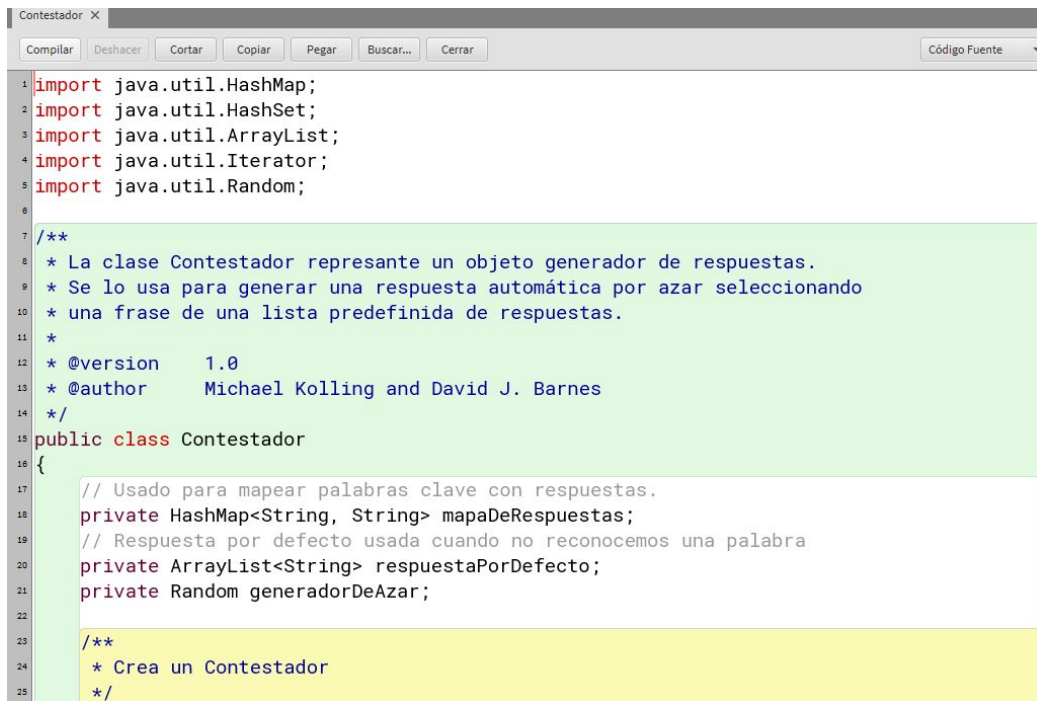
- Son **colecciones** que contienen pares de valores
 - Doblemente parametrizadas
- Almacenan entradas (entry) que asocian una **clave** y un **valor**
- Permiten recuperar el **valor** asociado a cada **clave**
- Se puede recorrer iterando sobre las entradas de la tabla
- Los métodos más relevantes son
 - **put, get, remove, keySet, values**

```
HashMap<String,Circulo> m;  
m = new HashMap<String,Circulo>()
```

```
m.put("cir1",new Circulo());  
m.put("otro",new Circulo());  
m.put("final",new Circulo());
```



Ejemplo de uso - Nuevo *Contestador*



```
1 import java.util.HashMap;
2 import java.util.HashSet;
3 import java.util.ArrayList;
4 import java.util.Iterator;
5 import java.util.Random;
6
7 /**
8  * La clase Contestador repesante un objeto generador de respuestas.
9  * Se lo usa para generar una respuesta automática por azar seleccionando
10  * una frase de una lista predefinida de respuestas.
11  *
12  * @version    1.0
13  * @author     Michael Kolling and David J. Barnes
14  */
15 public class Contestador
16 {
17     // Usado para mapear palabras clave con respuestas.
18     private HashMap<String, String> mapaDeRespuestas;
19     // Respuesta por defecto usada cuando no reconocemos una palabra
20     private ArrayList<String> respuestaPorDefecto;
21     private Random generadorDeAzar;
22
23     /**
24      * Crea un Contestador
25      */
26 }
```

Private vs. Public

- Los atributos, constructores y métodos **private** sólo son visibles y accesibles **sólo** dentro de la misma clase.
 - Los atributos, constructores y métodos **public** son visibles y accesibles desde otras clases.
 - Sólo deberían ser public los métodos que son para el uso de otras clases
 - Los atributos no deberían ser **public**, debería mantenerse ocultos para el resto de las clases y solo accederse a través de sus getters y setters.
 - El ocultamiento de los detalles de implementación incrementa la independencia
- 

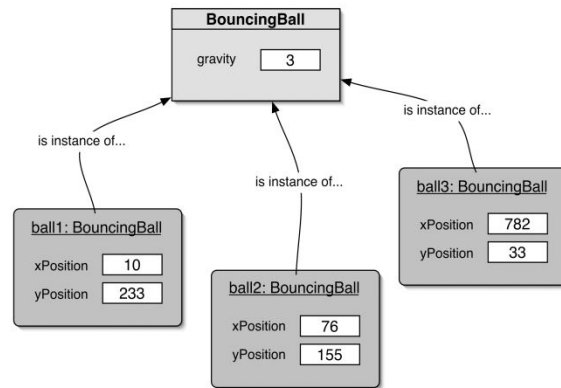
Variables de Clase

- Pertenecen a la clase (no a cada instancia)
 - no existe una variable por cada instancia
 - todas las instancias pueden acceder a la única copia de variable de clase que existe
 - estas variables existen aunque no existan instancias de la clase
- Se declaran agregando la palabra clave **static**
- Si además son **final**, no se pueden modificar
 - en este caso, pueden usarse como **constantes**
 - por convención, se sugiere que el nombre de las constantes sea todo en mayúsculas
- **Nota:** un método también puede ser **static** -> no requiere una instancia para invocarse! (ergo, no puede operar sobre variables de instancia).

```
public class ReboteDePelota
{
    private static final int GRAVEDAD = 3;

    private int diametro;
    private int xPosicion;
    private int yPosicion;

    private final int posicionDelPiso;
    private int desaceleracionDePelota = 2;
```



: Lote

int numero	1
String desc	Peugeto
Oferta ofertaMax	null

visorDeR1: VisorDeReloj

private VisorDeNumeros horas	
private VisorDeNumeros minutos	
private String cadVisor	

:ItemDeCorreo

de	Ana
para	Juan
mensaje	Hola

